

Entrega UT9

Todas las actividades tratan sobre un mismo proyecto, así que solo pasaré el proyecto y en este pdf iré mostrando los cambios que voy haciendo para cada actividad.

Actividad 1:

Diseño de la base de datos del colegio

El alumno debe:

- Diseñar una base de datos relacional con al menos dos tablas: Alumno y Curso.
- Definir claves primarias, foráneas y tipos de datos adecuados.
- Crear la base de datos en MySQL.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.mycompany</groupId>
  <artifactId>Actividad1</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>
  <dependencies>
    <!-- https://mvnrepository.com/artifact/mysql/mysql-connector-java -->
    <dependency>
      <groupId>mysql</groupId>
      <artifactId>mysql-connector-java</artifactId>
      <version>8.0.33</version>
    </dependency>
  </dependencies>
  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.release>23</maven.compiler.release>
    <exec.mainClass>com.mycompany.ColegioDb.ColegioDb</exec.mainClass>
  </properties>
  <name>ColegioDb</name>
</project>
```

```
package com.mycompany.ColegioDb;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;
```

```

public class BBDDManager
{
    private static String datosConexion="jdbc:mysql://localhost:3306/";
    private static String baseDatos = "Colegio";
    private static String usuario = "root";
    private static String password = "Jose0831";
    private Connection con;

    public BBDDManager()
    {
        try
        {
            con = DriverManager.getConnection(datosConexion, usuario, password);
            try
            {
                crearDb();

                con.close();
                con = DriverManager.getConnection(
                    datosConexion + baseDatos, usuario, password
                );

                crearTablaCurso();
                crearTablaAlumno();

            }
            catch(Exception e)
            {
                e.printStackTrace();
            }
        }
        catch(Exception e)
        {
            e.printStackTrace();
        }
    }
}

```

```

private void crearDb() throws Exception
{
    String query = "create database if not exists " + baseDatos + ";";
    Statement stmt = null;

    try
    {
        stmt = con.createStatement();
        stmt.executeUpdate(query);
    }
    catch(SQLException e)
    {
        e.printStackTrace();
    }
    finally
    {
        if (stmt != null)
        {
            stmt.close();
        }
    }
}

private void crearTablaAlumno() throws Exception
{
    String query = "CREATE TABLE IF NOT EXISTS Alumno ("
        + "idAlumno INT AUTO_INCREMENT PRIMARY KEY, "
        + "idCurso INT, "
        + "nombre VARCHAR(100) NOT NULL, "
        + "genero VARCHAR(100) NOT NULL, "
        + "edad INT NOT NULL, "
        + "CONSTRAINT fk_alumno_curso FOREIGN KEY (idCurso) REFERENCES "
        + "Curso(idCurso)"
        + ");";

    Statement stmt = null;

    try
    {
        stmt = con.createStatement();
        stmt.executeUpdate(query);
    }
    catch(SQLException e)
    {
        e.printStackTrace();
    }
    finally
    {
        if (stmt != null)
        {
            stmt.close();
        }
    }
}

```

```

private void crearTablaCurso() throws Exception
{
    String query = "CREATE TABLE IF NOT EXISTS Curso ("
        + "idCurso INT AUTO_INCREMENT PRIMARY KEY, "
        + "nombre VARCHAR(100) NOT NULL"
        + ");";
    Statement stmt = null;

    try
    {
        stmt = con.createStatement();
        stmt.executeUpdate(query);
    }
    catch(SQLException e)
    {
        e.printStackTrace();
    }
    finally
    {
        if(stmt != null)
        {
            stmt.close();
        }
    }
}

```

Actividad 2:

Conexión desde Java a la base de datos

El alumno debe:

- Configurar la conexión JDBC
- Probar la conexión con un mensaje de éxito o error.
- Usar DriverManager, Connection y SQLException

```

public class BBDDManager
{
    private static String datosConexion="jdbc:mysql://localhost:3306/";
    private static String baseDatos = "Colegio";
    private static String usuario = "root";
    private static String password = "Jose0831";
    private Connection con;
}

```

```

public BBDDManager()
{
    try
    {
        con = DriverManager.getConnection(datosConexion, usuario, password);
        System.out.println("Conectado al servidor MySQL");

        try
        {
            crearDb();

            con.close();
            con = DriverManager.getConnection(
                datosConexion + baseDatos, usuario, password
            );
            System.out.println("Conectado a la base de datos " + baseDatos + " correctamente");

            crearTablaCurso();
            crearTablaAlumno();

        }
        catch (Exception e)
        {
            System.out.println("Ha habido un error al conectarse a "
                + baseDatos + "\n" + e.getMessage());
        }
        catch (Exception e)
        {
            e.printStackTrace();
        }
    }
}

```

```

private void crearDb() throws Exception
{
    String query = "create database if not exists " + baseDatos + ";";
    Statement stmt = null;

    try
    {
        stmt = con.createStatement();
        stmt.executeUpdate(query);
    }
    catch (SQLException e)
    {
        e.printStackTrace();
    }
    finally
    {
        if (stmt != null)
        {
            stmt.close();
        }
    }
}

```

```

private void crearTablaAlumno() throws Exception
{
    String query = "CREATE TABLE IF NOT EXISTS Alumno ("
        + "idAlumno INT AUTO_INCREMENT PRIMARY KEY, "
        + "idCurso INT, "
        + "nombre VARCHAR(100) NOT NULL, "
        + "genero VARCHAR(100) NOT NULL, "
        + "edad INT NOT NULL, "
        + "CONSTRAINT fk_alumno_curso FOREIGN KEY (idCurso) REFERENCES "
        + "Curso(idCurso)"
        + ");";

```

```
Statement stmt = null;
```

```

try
{
    stmt = con.createStatement();
    stmt.executeUpdate(query);
}
catch(SQLException e)
{
    e.printStackTrace();
}
finally
{
    if(stmt != null)
    {
        stmt.close();
    }
}
}

```

```

private void crearTablaCurso() throws Exception
{
    String query = "CREATE TABLE IF NOT EXISTS Curso ("
        + "idCurso INT AUTO_INCREMENT PRIMARY KEY, "
        + "nombre VARCHAR(100) NOT NULL"
        + ");";
    Statement stmt = null;

    try
    {
        stmt = con.createStatement();
        stmt.executeUpdate(query);
    }
    catch(SQLException e)
    {
        e.printStackTrace();
    }
    finally
    {
        if(stmt != null)
        {
            stmt.close();
        }
    }
}

```

Prueba:

```

/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to ch
 */

package com.mycompany.ColegioDb;

/**
 *
 * @author josee
 */
public class ColegioDb {

    public static void main(String[] args)
    {
        new BBDDManager();
    }
}

```

```

--- resources:3.3.1:resources (default-resources) @ Actividad1 ---
skip non existing resourceDirectory C:\Users\josee\Desktop\Github\Programming\LinkiaGradoA\ol\Progr

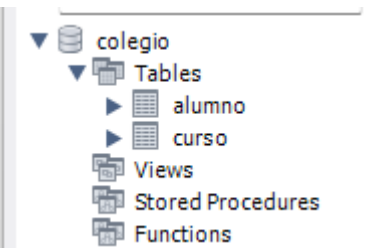
--- compiler:3.13.0:compile (default-compile) @ Actividad1 ---
Recompiling the module because of changed source code.
Compiling 2 source files with javac [debug release 23] to target\classes

--- exec:3.1.0:exec (default-cli) @ Actividad1 ---
Conectado al servidor MySQL
Conectado a la base de datos Colegio correctamente

-----
BUILD SUCCESS
-----

Total time: 1.517 s
Finished at: 2025-12-18T22:06:17+01:00
-----

```



Actividad 3:

Implementar operaciones CRUD para alumnos

El alumno debe:

- Crear métodos en Java para insertar, modificar, eliminar y consultar alumnos.
- Usar PreparedStatement para evitar inyecciones SQL.
- Mostrar los resultados por consola.

Cada alumno tiene foreign key de un curso, así que también he decidido crear los métodos para curso.

```

public void insertarCurso(String nombre)
{
    String query = "INSERT INTO Curso (nombre) VALUES (?)";

    try(PreparedStatement pstmt = con.prepareStatement(query))
    {
        pstmt.setString(1, nombre);
        pstmt.executeUpdate();

        System.out.println("Curso insertado");
    }
    catch(SQLException e)
    {
        e.printStackTrace();
    }
}

```



```
public void modificarCurso(int idCurso, String nombre)
{
    String query = "UPDATE Curso SET nombre = ? WHERE idCurso = ?";

    try(PreparedStatement pstmt = con.prepareStatement(query))
    {
        pstmt.setString(1, nombre);
        pstmt.setInt(2, idCurso);
        pstmt.executeUpdate();

        System.out.println("Curso modificado");
    }
    catch(SQLException e)
    {
        e.printStackTrace();
    }
}

public void eliminarCurso(int idCurso)
{
    String query = "DELETE FROM Curso WHERE idCurso = ?";

    try(PreparedStatement pstmt = con.prepareStatement(query))
    {
        pstmt.setInt(1, idCurso);
        pstmt.executeUpdate();

        System.out.println("Curso eliminado");
    }
    catch(SQLException e)
    {
        e.printStackTrace();
    }
}
```

```

public void mostrarCursos ()
{
    String query = "SELECT * FROM Curso";

    try(PreparedStatement pstmt = con.prepareStatement(query);
        ResultSet rs = pstmt.executeQuery())
    {
        System.out.println("Cursos:");
        while(rs.next())
        {
            int idCurso = rs.getInt("idCurso");
            String nombre = rs.getString("nombre");

            System.out.println("ID: " + idCurso + ", Nombre: " + nombre);
        }
    }
    catch(SQLException e)
    {
        e.printStackTrace();
    }
}

```

```

public void insertarAlumno(int idCurso,String nombre, String genero, int edad)
{
    String query = "INSERT INTO Alumno (idCurso, nombre, genero, edad) VALUES "
        + "(?, ?, ?, ?)";

    try(PreparedStatement pstmt = con.prepareStatement(query))
    {
        pstmt.setInt(1, idCurso);
        pstmt.setString(2, nombre);
        pstmt.setString(3, genero);
        pstmt.setInt(4, edad);
        pstmt.executeUpdate();

        System.out.println("Alumno insertado");
    }
    catch(SQLException e)
    {
        e.printStackTrace();
    }
}

```

```

public void modificarAlumno(int idAlumno, int idCurso, String nombre, String genero, int edad)
{
    String query = "UPDATE Alumno SET idCurso= ?, nombre = ?, genero = ?,"
        + "edad = ? WHERE idAlumno = ?";

    try(PreparedStatement pstmt = con.prepareStatement(query))
    {
        pstmt.setInt(1, idCurso);
        pstmt.setString(2, nombre);
        pstmt.setString(3, genero);
        pstmt.setInt(4, edad);
        pstmt.setInt(5, idAlumno);
        pstmt.executeUpdate();

        System.out.println("Alumno modificado");
    }
    catch(SQLException e)
    {
        e.printStackTrace();
    }
}

```

```

public void eliminarAlumno(int idAlumno)
{
    String query = "DELETE FROM Alumno WHERE idAlumno = ?";

    try(PreparedStatement pstmt = con.prepareStatement(query))
    {
        pstmt.setInt(1, idAlumno);
        pstmt.executeUpdate();

        System.out.println("Alumno eliminado");
    }
    catch(SQLException e)
    {
        e.printStackTrace();
    }
}

```

```

public void mostrarAlumnos ()
{
    String query = "SELECT * FROM Alumno";

    try (PreparedStatement pstmt = con.prepareStatement(query);
        ResultSet rs = pstmt.executeQuery())
    {
        System.out.println("Alumnos:");
        while (rs.next())
        {
            int idAlumno = rs.getInt("idAlumno");
            int idCurso = rs.getInt("idCurso");
            String nombre = rs.getString("nombre");
            String genero = rs.getString("genero");
            int edad = rs.getInt("edad");

            System.out.println("ID: " + idAlumno
                               + ", idCurso: " + idCurso
                               + ", Nombre: " + nombre
                               + ", Genero: " + genero
                               + ", Edad: " + edad);
        }
    }
    catch (SQLException e)
    {
        e.printStackTrace();
    }
}

```

```

//
public class ColegioDb {

    public static void main(String[] args)
    {
        BBDDManager bbddManager = new BBDDManager();

        bbddManager.insertarCurso("Matemáticas");
        bbddManager.mostrarCursos();

        bbddManager.insertarAlumno(1, "Juan", "M", 18);
        bbddManager.insertarAlumno(1, "Maite", "F", 23);

        bbddManager.modificarAlumno(1, 1, "Jose", "M", 26);

        bbddManager.eliminarAlumno(2);
        bbddManager.mostrarAlumnos();
    }
}

```

Resultado por consola:

```
Conectado al servidor MySQL
Conectado a la base de datos Colegio correctamente
Curso insertado
Cursos:
ID: 1, Nombre: Matemáticas
Alumno insertado
Alumno insertado
Alumno modificado
Alumno eliminado
Alumnos:
ID: 1, idCurso: 1, Nombre: Jose, Genero: M, Edad: 26
```

The screenshot shows a MySQL GUI window with a query editor and a result grid. The query editor displays the following SQL query:

```
1 • SELECT * FROM colegio.alumno;
```

The result grid shows the following data:

	idAlumno	idCurso	nombre	genero	edad
▶	1	1	Jose	M	26
*	NULL	NULL	NULL	NULL	NULL

The GUI also includes a toolbar with various icons for file operations, a 'Limit to 1000 rows' dropdown, and a 'Result Grid' tab. The 'Filter Rows' field is empty, and the 'Edit' button is visible.

1 • `SELECT * FROM colegio.curso;`

Result Grid

Filter Rows:

Edit:

Export/Import

	idCurso	nombre
▶	1	Matemáticas
✱	NULL	NULL